조건문과 반복문 실습

컴퓨터학부 박한영

비교/논리/삼항 연산자

- 연산자의 종류
  - 비교 연산자: ==(동등), !=(다름), >, >=, <, <=
  - 논리 연산자: &&(AND), ||(OR), !(NOT)
  - 삼항 연산자: (조건식) ? (참일 때) : (거짓일 때)

- C에서의 논리 연산은 그 결과가 참일 경우 1(true), 거짓일 경우 0(false)를 반환
- 조건문의 조건식에 다양한 연산자를 활용하여 코드를 간략화할 수 있음!

```c
if(hour >= 9) {
    if(hour <= 10) {
        printf("1교시");
    }
}
```

```c
if(hour >= 9 && hour <= 10) {
    printf("1교시");
}
```

비교/논리/삼항 연산자

- 조건식을 쓸 때 하는 실수들
  - 60 <= score <= 80 (x)
    - ➢ 둘 이상의 비교 연산은 반드시 논리 연산자로 묶어야 함!
  - 60 <= score & score <= 80
    - ➢ 연산이 가능하나, &는 비트 연산자이기에 모든 비교 연산을 진행함!
  - if(score = 60)
    - ➢ = 는 대입 연산자이고, 위의 경우 반환값은 대입값이므로 항상 참이됨!

C언어의 중괄호

- C언어는 중괄호를 통해 코드 블록을 구분함
- 조건문과 반복문의 경우 뒤에 있는 중괄호 블록 전체를 루프에서 실행하게 됨.
- 중괄호 내부에서 선언된 변수의 경우 해당 영역에서만 사용 가능

```c
if(param % 2 == 0)
    printf("%d\n", param);
    printf("2의배수");
if(param % 3 == 0)
    printf("%d\n", param);
    printf("3의배수");
```

```c
if(param % 2 == 0) {
    printf("%d\n", param);
    printf("2의배수");
}
if(param % 3 == 0) {
    printf("%d\n", param);
    printf("3의배수");
}
```

if문

- 조건식이 참이면 주어진 명령문을 수행
- if, else if, else로 나눌 수 있음
- 명령문이 한 줄일 경우 중괄호 생략 가능!

```c
// 1. if 단일
if(condition)
    statement;

// 2. if | else
if(condition)
    statement;
else
    statement;

// 3. if | else if | else
if(condition)
    statement;
else if(condition)
    statement;
else
    statement;
```

```c
if(score >= 90) {
    printf("A입니다.");
}
else if(score >= 80) {
    printf("B입니다.");
}
else if(score >= 70) {
    printf("C입니다.");
}
else if(score >= 60) {
    printf("D입니다.");
}
else {
    printf("F입니다.");
}
```

for문

- 초기식; 조건식; 증감식의 구조를 가지고 있는 반복문
- for 루프의 초기식과 조건식, 증감식에는 콤마를 활용하여 여러 개의 식을 넣을 수 있음

for (초기식; 조건식; 증감식) { 명령문; }

- ➢ 초기식: for문 들어가기 전 한 번만 실행
- ➢ 조건식: 매 루프마다 조건식을 검사하고, 참인 경우 명령문을 수행
- ➢ 증감식: 루프가 끝날 때마다 실행됨

```
for(int i = 0; i < n; i++)
    printf("%d\n", i);
```

while문

- 조건식이 참인 동안 명령문을 수행하고, 루프가 끝날 때마다 다시 조건식을 검사하여 반복하는 구조
- while(i++) 과 같은 구조로 조건식에 증감 연산을 할 수도 있음

while(조건식) { 명령문; }

➢ 조건식: 매 루프마다 조건식을 검사하고, 참인 경우 명령문을 수행

```
int n;
scanf("%d", &n);
while(n < 100) {
    n *= 2;
    printf("%d\n", n);
}
```

do-while문

- while문과 동일하게 조건식이 참인 동안 명령문을 수행하지만, 맨 처음의 명령문 실행을 보장하는 것이 차이!

```
do {
    printf("처음은 무조건 실행");
} while(0);
```

do { 명령문; } while(조건식);

➢ 조건식: 일단 명령문을 한 번 실행하고, 이후 매 루프마다 조건식을 검사

무한 반복

- 조건식이 항상 참이면 무한으로 반복함
- 현대의 컴파일러에서는 둘 다 차이가 없음

for(;;) statement;

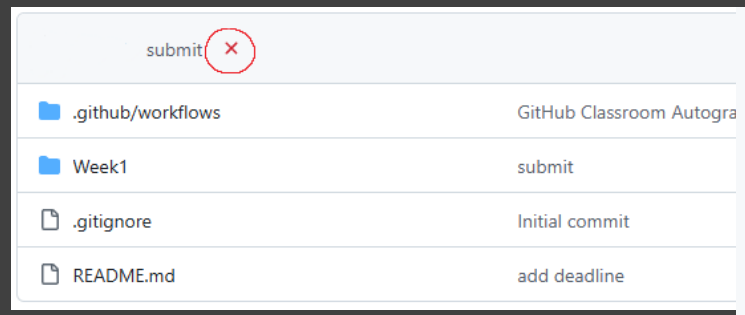
- 초기식, 조건식, 증감식이 모두 없음
- 조건은 항상 참이고, 증감이 일어나지 않으므로 무한히 반복됨

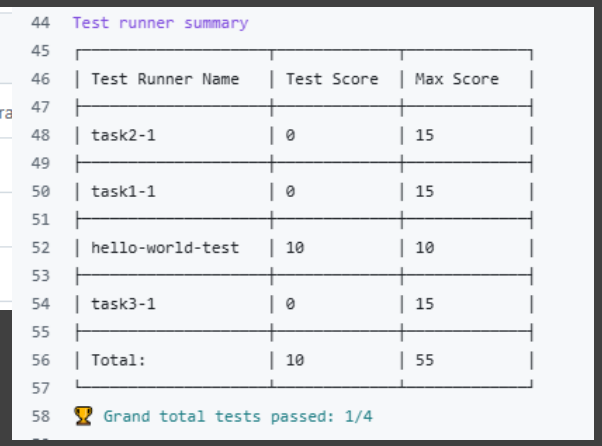
while(1) statement;

- 조건식에 참 값인 1이 들어가 있으므로 항상 참이 되어 무한히 반복됨

과제 안내

- 링크로 들어가 과제 수락
- 과제 수락 후 본인의 과제 레포지토리가 생긴 걸 확인 후 clone하여 작업(첫 실습 ppt 참고)
- 과제를 완료하면 git push 후 본인의 레포지토리에서 결과 확인 (아래 그림 참조)
- 동그라미 친 x(과제 만점 시 체크표시로 바뀜)를 누르고 details를 누르면 아래와 같이 상세 점수를 확인할 수 있음.





```
Test runner summary
┌──────────────────┬──────────────┬──────────────┐
│ Test Runner Name │ Test Score   │ Max Score    │
├──────────────────┼──────────────┼──────────────┤
│ task2-1          │ 0            │ 15           │
├──────────────────┼──────────────┼──────────────┤
│ task1-1          │ 0            │ 15           │
├──────────────────┼──────────────┼──────────────┤
│ hello-world-test │ 10           │ 10           │
├──────────────────┼──────────────┼──────────────┤
│ task3-1          │ 0            │ 15           │
├──────────────────┼──────────────┼──────────────┤
│ Total:           │ 10           │ 55           │
└──────────────────┴──────────────┴──────────────┘
🏆 Grand total tests passed: 1/4
```

실습 안내

- **문제 1.**
- 어떤 연도를 입력받아 그 해가 윤년인지 판단하시오.
- 4로 나누어 떨어지고 / 100으로는 나누어 떨어지지 않되, 400으로 나누어 떨어지는 해는 윤년이다.
- 입력: 정수인 연도 하나가 입력된다.
- 출력: 만약 윤년이면 "Leap Year", 평년이면 "Common Year"를 출력한다.

```c
#include <stdio.h>

int main(void) {
    int year;

    /*
    write your codes here.
    */

    return 0;
}
```

```
input: 2000
output: Leap Year
input: 1900
output: Common Year
input: 2024
output: Leap Year
```

실습 안내

- **문제 2.**
- 세 정수 A, B, C를 입력 받고 A부터 B까지의 정수 중 C의 배수인 수의 합을 출력하시오.
- 입력: 두 자연수 A, B, C (1 ≤ A ≤ C ≤ B ≤ 10,000,000)
- 출력: 계산한 값의 합계를 출력한다.

```c
#include <stdio.h>

int main(void) {
    int A, B, C;
    int sum = 0;

    /*
    write your codes here.
    */

    return 0;
}
```

```
input: 1 10 2
output: 30 | (2+4+6+8+10)
input: 3 15 4
output: 24 | (4+8+12)
input: 5 20 5
output: 50 | (5+10+15+20)
```

실습 안내

- **문제 3.**
- 연산 기호 op와 연산을 수행할 정수 A, B를 종료 조건 전까지 계속 입력 받아 결과를 출력한다.
- 입력: 연산 기호 op, 정수 A, B. (힌트: op를 먼저 입력받고 분기 수행 후 A, B 입력받기)
- 출력: op가 연산자(+-*/%)인 경우 계산 수행, #인 경우 루프 종료, 그 외의 기호의 경우 메시지 "Error" 출력하기

```c
#include <stdio.h>

int main(void) {
    int A, B;
    char op;

    /*
    write your codes here.
    */

    return 0;
}
```

```
input:
+ 3 5
* 2 4
#
output:
8
8
```

```
input:
- 10 7
x 3 4
/ 8 2
+ 7 6
#
output:
3
Error
4
13
```

```
input:
!
#
output:
Error
```

과제 안내

- **문제 1.**
- 1이상의 정수 n이 주어질 때 n층의 피라미드를 출력하시오.
- 입력: 정수 n
- 출력: 별(*)로 이루어진 n층의 피라미드를 출력한다.

```c
#include <stdio.h>

int main() {
    int n;

    /*
    write your codes here.
    */

    return 0;
}
```

```
input: 7
output:
      *
     ***
    *****
   *******
  *********
 ***********
*************
```

과제 안내

- **문제 2.**
- 두 자연수 A와 B가 주어질 때, 두 수가 서로소인지 판별하시오.
- 입력: 두 자연수 A, B (1≤ A, B ≤ 100,000)
- 출력: 두 자연수가 서로소이면 Yes, 그렇지 않으면 No를 출력한다.

```c
#include <stdio.h>

int main() {
    int A, B;

    /*
    write your codes here.
    */

    return 0;
}
```

```
input: 6 12
output: No
input: 14 25
output: Yes
```

과제 안내

- **문제 3.**
- 1 이상의 정수 하나를 받아 소인수분해 한 결과를 출력한다. (효율적인 알고리즘 고려 x)
- 입력: 소인수분해 할 정수 n이 주어진다.
- 출력: 입력받은 숫자를 소인수분해 한 결과를 공백으로 구분하여 출력한다. (작은 숫자부터 출력)

```c
#include <stdio.h>

int main() {
    int n;

    /*
    write your codes here.
    */

    return 0;
}
```

```
input: 12
output: 2 2 3
input: 37
output: 37
input: 72
output: 2 2 2 3 3
```

THANK YOU!